

Prolog

Aritmética

- Introducir predicados predefinidos para operaciones aritméticas
- Programas simples, uso de acumuladores
- Consideraciones de eficiencia, *tail recursion*

Aritmética en Prolog

- $2 + 3 = 5$
- $3 \times 4 = 12$
- $5 - 3 = 2$
- $3 - 5 = -2$
- $8 / 2 = 4$
- 2 es el resto de dividir 8 entre 3

- 5 is $2 + 3$
- 12 is 3×4
- 2 is $5 - 3$
- -2 is $3 - 5$
- 4 is $8 / 2$
- 2 is $8 \bmod 3$

Ejemplos de consultas - is

?- 10 is 5 + 5.

yes

?- 4 is 2 + 3.

no

?- X is 3 * 4.

X=12

yes

Ejemplos de consultas is vs. unificación

?- 10 is 5 + 5.

yes

?- 4 is 2 + 3.

no

?- X is 3 * 4.

X=12

yes

?- 10 = 5 + 5.

no

?- 4 = 2 + 3.

no

?- X = 3 * 4.

X= 3 * 4

yes

Predicado del sistema *is*

- *is* (Valor, Expresion)
is (3,2+1) es verdadero

Predicado del sistema *is*

- *is* (Valor, Expresion)
is (3,2+1) es verdadero
- Notación infija: 3 *is* 2+1

Predicado del sistema *is*

- *is* (Valor, Expresion)
is (3,2+1) es verdadero
- Notación infija: 3 *is* 2+1
- Se evalúa Expresión y se unifica con Valor

Predicado del sistema *is*

- Una consulta en Prolog puro puede ser exitosa o fallar.
- Con predicados del sistema puede darse además la situación de ***error***
- ***is*** exige que la expresión a evaluar tenga el tipo adecuada : números o variables instanciadas en valores numéricos

Otros predicados del sistema para aritmética y comparaciones

$<$, $=<$, $>=$, $:=$, $\neq$

- Son operadores infijos: ej. $\text{Expr1} < \text{Expr2}$
- Realizan evaluación de ambas expresiones
- Toda variable debe estar instanciada

Largo de una lista

- Inducción en la estructura

Largo de una lista

- Inducción en la estructura
- La lista vacía tiene largo 0.

Largo de una lista

- Inducción en la estructura
- La lista vacía tiene largo 0.
- El largo de la lista $[H|T]$ es el largo de la lista T más 1.

Largo de una lista

`largo([],0).`

`largo([H|T],L) :-`

`largo(T,L1),`

`L is L1+1.`

Largo de una lista

`largo([],0).`

`largo([H|T],L) :-`

`largo(T,L1),`

`L is L1+1.`

- `?- largo([a,b,c],L).` árbol de búsqueda

Largo de una lista

¿Cómo sería largo con un acumulador?

Largo de una lista

¿Cómo sería largo con un acumulador?

```
largo([_|T],Ac,L):-
```

```
    Ac1 is Ac+1,
```

```
    largo(T,Ac1,L).
```

```
largo([],L,L).
```

Largo de una lista

Encapsulamos largo con acumulador:

```
largo(Lista, Largo) :- largo(Lista,0,Largo).
```

```
largo([_|T],Ac,L):-
```

```
    Ac1 is Ac+1,
```

```
    largo(T,Ac1,L).
```

```
largo([],L,L).
```

Comparando ambas versiones de largo

- Tail recursion – la invocación recursiva se hace en el último predicado del lado derecho.
 - Mayor eficiencia, no hay que hacer cálculos ni guardar valores de variables en el stack de ejecución
 - Utilizando acumuladores puede ser más fácil obtener un programa *tail-recursive*

Máximo de una lista

- Calculamos el máximo de una lista de enteros
- 2 versiones, una de ellas con acumuladores

Máximo de una lista

```
maxL([H|T],M):-  
    maxL(T,M1),  
    max(H,M1,M).  
maxL([X],X).
```

```
max(A,B,A):-A>=B.  
max(A,B,B):-A<B.
```

Máximo de una lista, acumulador

El acumulador en este caso lleva el máximo calculado hasta el momento.

maxL([H|T],Max,M):-

max(H,Max,Max1).

maxL(T,Max1,M),

maxL([],X,X).

max(A,B,A):-A>=B.

max(A,B,B):-A<B.

Ejercicios

1) Vectores :

1) producto de un escalar por un vector

2) Producto interno de 2 vectores

2) $\text{entre}(I, J, K) \leftarrow I, J, K \text{ naturales, } I \leq K \leq J$

3) $\text{rango}(I, J, L) \leftarrow$ la lista L contiene los naturales entre los naturales I y J , en ese orden.

Resumen

- Manejo de aritmética en Prolog
- Falla y error de ejecución para predicados Prolog
- Tail recursion, vinculación entre tail recursion y técnica de acumuladores

Próxima

Fundamentos teóricos (4 clases)

- Lógica de 1er orden
- Cláusulas, cláusulas de Horn
- Sustituciones, unificación. mgu
- Inferencias, derivación, consecuencia lógica
- Resolución, caso proposicional, refinamientos de la resolución, resolución SLD

Referencias

Basado en las transparencias de:

Blackburn, P, J. Bos y K. Striegnitz. 2001. *Learn Prolog Now* (<http://www.learnprolognow.org>).